

Corrisio
Rafa

75/100

Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	NºOrden
		

Turno: ☒ Mañana

☐ Tarde/Noche

Materia: ☒ TDA

☐ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

1. Preguntas de opción multiple

Cada pregunta tiene 4 opciones y una sola opción correcta. Las respuestas correctas (Bien) suman 5 puntos. Las respuestas incorrectas (Mal) restan 2 puntos. Las preguntas no respondidas no se consideran como incorrectas y suman 0 puntos. Marcar en el cuadrado la opción que consideren correcta.

Pregunta 1 Sea la siguiente función recursiva que calcula el número combinatorio $\binom{N}{K}$

$$\binom{N}{K} = \begin{cases} 1, & \text{si } k = 0 \vee k = n \\ 0, & \text{si } k < 0 \vee k > n \\ \binom{N-1}{K-1} + \binom{N-1}{K} & \text{si no} \end{cases}$$

¿Cuál es la complejidad temporal de calcular el número $\binom{N}{K}$ utilizando Programación Dinámica?

☐ $\Theta(N^2)$

☐ $\Theta(K^2)$

☒ $\Theta(N * K)$

☐ $\Theta(N^2 * K)$

Pregunta 2 Dado un grafo con N nodos y M aristas. ¿Cuál es la complejidad espacial de la representación como matriz de adyacencia?

☒ $\Theta(N^2)$

☐ $\Theta(N + M)$

☐ $\Theta(N * M)$

☐ $\Theta(M)$

Pregunta 3 Si en la representación de un grafo de N nodos (1 a N) y M aristas como lista de adyacencia, reemplazo los vectores que contienen a los vecinos de cada nodo por tablas hash (*) (ej: unordered_set en C++, HashSet en Rust, dict en Python), podré:

☐ Poder recorrer los vecinos de un nodo v dado en orden ascendente en $O(d(v))$

☐ Tener complejidad espacial $O(N)$

☐ Saber si dos nodos u y v son vecinos en $\Theta(1)$

☐ Saber si dos nodos u y v tienen vecinos en común en $O(\log(N))$

(*) Durante el parcial no se responderán consultas sobre lo que es una tabla hash.

Pregunta 4 Dado el dígrafo con 5 nodos y aristas: (1, 2), (1, 3), (2, 4), (3, 4), (4, 5). ¿Cuál de los siguientes es un recorrido de DFS válido?

☒ 1-2-4-5-3

☐ 1-2-4-3-5

☐ 1-2-3-4-5

☐ 1-3-2-4-5

→ puede ser pseudografo

Pregunta 5 Dado un grafo dirigido con nodos del 1 al N , donde el nodo i ($1 \leq i \leq N$) tiene asociado un string s_i (todos los s_i son no vacíos y distintos) en el cual hay una arista (i, j) si y solo si s_j es un sufijo de s_i . ¿Cuál de las siguientes afirmaciones es falsa?

→ si es sufijo de s_i

- ☐ De un nodo i pueden salir a lo sumo $|s_i| * \text{aristas}$
- ☐ El grafo tiene bucles (pares (i, i))
- ☐ El grafo es transitivo
- ☒ El grafo puede tener ciclos dirigidos simples con más de un nodo

(*) $|s|$ es la cantidad de caracteres de s

Pregunta 6 Si represento a una función recursiva f (que al computarla no genera una recursión infinita) como un grafo dirigido donde cada uno de los N nodos es un estado y una arista (a, b) significa "La función se llama a sí misma con el estado b para procesar el estado a ". ¿Cuál de las siguientes afirmaciones es correcta?

- ☐ Puede haber ciclos en el grafo
- ☐ Siempre podemos calcular la función f para todos los estados en $O(N)$
- ☐ Los casos base tienen grado de entrada 0
- ☒ Llamar a la función con el estado x generará tantas llamadas al estado y como caminos hay de x a y en el grafo

Pregunta 7 La relación entre los nodos de un grafo no dirigido definida como $R(a, b)$ si y sólo si a y b pertenecen a la misma componente conexa, es:

- ☒ Una relación de equivalencia
- ☐ Una relación de orden (parcial o total)
- ☐ Anti-simétrica ($R(a, b) \implies \neg R(b, a)$)
- ☐ Una relación irreflexiva (no existe $R(a, a)$)

Pregunta 8 La complejidad espacial de la memorización de una DP con N estados y M transiciones entre estados, implementada de forma top-down recursiva, es:

- ☒ $\Theta(M)$
- ☐ $\Theta(N + M)$
- ☒ $\Theta(N)$
- ☐ $\Theta(N * M)$

2. Preguntas abiertas de respuesta corta

Respuesta Bien: +5 puntos. Respuesta Regular, Mal o sin completar: +0 puntos.

Pregunta 9 En el algoritmo de DP de Floyd Warshall visto en clase, ¿Qué se calcula en cada estado de la DP?

B El estado está determinado por i, j y k , y calcula el camino mínimo de i a j tomando sólo los nodos intermedios desde 1 hasta k .
siendo i, j nodos

Pregunta 10 En el algoritmo de DP de Bellman Ford visto en clase, ¿Qué se calcula en cada estado de la DP?

R El estado está determinado por i , y calcula el camino mínimo desde el origen hasta el nodo i . Solo vale en la última iteración

Pregunta 11 Sea G un grafo dirigido (dígrafo) donde los nodos son los números de 1 a N y existe una arista (a, b) si y sólo si existe un número primo p tal que $b = a * p$. ¿Qué significa que exista un camino de u a v en G ?

B significa que $v = u * p_1 * p_2 * \dots * p_k$, es decir que v es un múltiplo de u y además, v es un producto de u por tantos primos como nodos intermedios tiene el camino (si pasé por $k-1$ nodos intermedios, además el nodo v).
tendré k primos multiplicando u .
→ ya que el nodo v "aporta" un primo también.

Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	N°Orden
		

Turno:

☒ Mañana

☐ Tarde/Noche

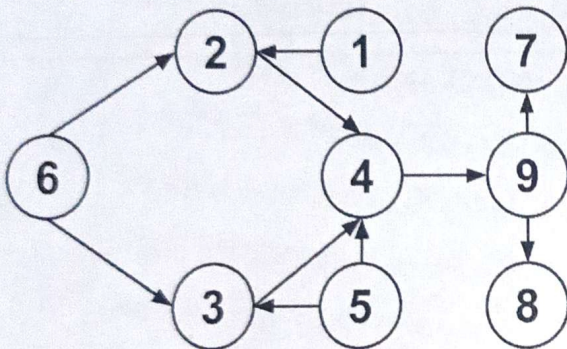
Materia:

☒ TDA

☐ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

Pregunta 12 Indique un orden topológico válido del siguiente grafo dirigido acíclico:



~~1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9~~ 6 → 1 → 5 → 2 → 3 → 4 → 9 → 7 → 8

El 6 se hace primero, luego el 1 y así sucesivamente siguiendo las flechas hasta el 8.

3. Ejercicios a Desarrollar

Ejercicio Bien: +10 puntos. Ejercicio Regular: +5 puntos. Ejercicio Mal o sin completar: +0 puntos.

Pregunta 13 Dado un grafo dirigido $G = (V, E)$ con nodos números de 1 a N . Se tiene asociado a cada nodo v un peso p_v entero positivo.

Se pide diseñar un algoritmo o utilizar uno existente que en $O(N^3)$ (espacial y temporal) determine cada par ordenado de nodos (u, v) si es posible llegar de u a v en el dígrafo dado y, si es posible, determinar el menor peso P tal que es posible llegar utilizando como nodos intermedios (sin contar u ni v) solo a nodos con un peso menor o igual a P .

En caso de utilizar una modificación a un algoritmo existente, solo es necesario detallar cuales serían estas modificaciones.

Pregunta 14 Dado un cubo de Rubik, definimos el estado del Cubo como una posición definida por el color de cada celda que hay en cada cara (cada cara tiene un identificador que permite diferenciarlas). El cubo se considera resuelto si cada cara tiene únicamente celdas de un mismo color. Una jugada válida consiste en rotar una cara del cubo 90 grados en sentido horario o anti-horario.

Se pide modelar el cubo con un grafo donde:

A) Cada nodo represente un estado del cubo.

B) La distancia entre dos nodos u y v sea igual a la cantidad de jugadas necesarias para convertir el estado representado por u en el estado representado por v .

Pregunta 15 Sea el problema Sudoku N el problema de rellenar una matriz de $N \times N$ con números de 1 a N tal que no haya dos celdas con el mismo número en la misma fila ni en la misma columna. Sean estas dos soluciones con Backtracking a dicho problema:

```

1 def Sudoku(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return int(es_valido(estado))
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         retorno += Sudoku(estado, celda + 1)
9         estado[fil][col] = 0
10    return retorno

```

```

1 def Sudoku_poda(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return 1
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         if es_valido(estado):
9             retorno += Sudoku_poda(estado, celda + 1)
10        estado[fil][col] = 0
11    return retorno

```

Demostrar que:

- A) La cantidad de estados visitados por Sudoku es $\Theta(N^{(N^2)})$
- B) La cantidad de estados visitados por Sudoku_poda es $O((N!)^N * N^2)$

Puede usar lo siguiente sin necesidad de demostrarlo:

$$\sum_{i=1}^{i=n} p^i \in \Theta(p^n)$$

Cada una de las 2 demostraciones puede estar Bien, Regular o Mal. Para tener el ejercicio Bien ambas deben estar Bien. Para tener el ejercicio Regular se necesita que ambas estén Regular o al menos una esté Bien. Caso contrario, se tendrá el ejercicio Mal.

Pregunta 16 Sea el problema:

Dado un conjunto S de N ítems (p_i, v_i) y un peso límite P , no negativo. Determinar el máximo valor posible V tal que existe un subconjunto S' de S tal que:

- A) La suma de los p_i con i en S' es menor o igual a P .
- B) La suma de los v_i con i en S' es igual a V .

Dicho de otra forma, el máximo valor total entre los subconjuntos cuyo peso total no excede P . Y dada la siguiente solución (S es dado como una lista):

```

1 def Mochila(P: int, S: list[tuple[int,int]]) -> int:
2     if P < 0: return -inf
3     if len(S) == 0: return 0
4     (peso, valor) = S[0]
5     return max(Mochila(P, S[1:]), Mochila(P-peso, S[1:])+valor)

```

No tomo el elemento 0. y el resto si.

Demostrar que $Mochila(P, S)$ (con los P y S del enunciado original) devuelve la respuesta al problema planteado, independientemente del orden de los elementos en S .

Un cubo rubik es ~~siempre~~ $n \times n$, pero lo estoy planteando de forma general ya que no dicen nada en el enunciado.

R14) Cada cara tiene un identificador y tomare en cuenta que dado el identificador puedo saber ^{cuál} ~~que~~ cara tengo a derecha, izquierda, arriba y abajo. (Si no puedo asumir esto, lo construiría yo misma de manera sencilla, ya que al tener un cubo Rubik cada cara va en un lugar específico y puedo para cada cara, guardar en una tupla (por ejemplo) la cara que tiene a der, izq, arriba y abajo respectivamente). Esto ya está hecho al momento de modelar mi grafo.

- Cada nodo está compuesto por un diccionario donde cada clave es una cara y cada valor respectivo es una matriz $n \times m$ donde n son mis filas y m las columnas (iguales dimensiones en todas las caras). ~~esto del $n \times m$ es solo para darle un nombre porque podría haber de muchas formas una fila puede ser una columna~~ matriz $[i][j]$ me dice el color que tengo en la celda i, j . ✓

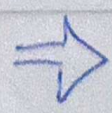
me lleva de v a v'

- Las aristas tienen peso 1 (se puede ver como un grafo no ponderado) ya que me indican una jugada válida.
 $\exists (v, v') \Leftrightarrow$ puedo hacer una transición (una jugada válida ~~de v a v'~~). El grafo no es dirigido, ya que puedo volver atrás y ~~repetir~~ repetir jugadas. ✓

la cara que está a la derecha de C_k .

- Transición: Puedo hacer 2 cosas a partir de un estado.
~~1) Dada 1 cara C_k (solo puedo modificar una cara por vez, y puedo elegir cualquier cara para modificar)~~
 (solo puedo hacer una jugada válida sobre una cara a la vez y puedo elegir cualquiera de las caras para esto):

1) Giro cara C_k hacia ^{derecha} ~~abajo~~ $\Rightarrow$ Modifico todas las caras abajo de ~~la cara C_k~~ la cara C_k abajo está y luego la cara ~~abajo la última que modifico y así hasta volver a C_k ya que voy todo la vuelta~~
~~minima de~~



Rotación

cara derecha a $C_k = der(C_k)$: $\forall j$ en rango:

~~der(C_k)[0][j] = der(C_k)[n-1][j]~~
 $der(C_k)[0][j] = \text{cara arriba de } der(C_k)[n-1][j]$

↓
cara arriba de ~~der(C_k)~~ $der(C_k)$.

luego repito esto para toda cara abajo de ~~der(C_k)~~ la que modifiqué anteriormente hasta llegar de vuelta al inicio ($der(C_k)$). Por último, en la cara que roté (C_k), cambio la primer fila por la última columna y así sucesivamente hasta que quede toda la cara rotada.

2) Roto cara C_k hacia izquierda: Es análogo al caso

anterior: cara izq a $C_k = izq(C_k)$: $\forall j$ en rango:

$izq(C_k)[n-1][j] = \text{arriba de } (izq(C_k))[n-1][j]$ y repito

para toda cara abajo de $izq(C_k)$ hasta volver al

inicio. Por último hago la rotación correspondiente

de C_k : primer fila pasa a primer columna y así sucesivamente.

B

R16) Quiero demostrar que Mochila (P, S) resuelve el problema sin importar el orden de los elementos de S.

~~Lo demuestro por inducción en la longitud de S.~~
~~Caso base: $n=0$~~

Primero veo el caso $P < 0$, en este caso no puedo tener una respuesta válida, ya que P no puede ser negativo en ningún momento, entonces está bien que devuelva $-\infty$ que es el neutro del máximo.

Ahora demuestro el resto con inducción en la longitud de S:

- Caso base = ~~$n=0$~~ $n = \text{len}(S) = 0$, si no tengo elementos, lo máximo que puedo sumar es 0, entonces la función es correcta en este caso.

- Paso inductivo: ~~Para toda longitud que sea mayor a 0 y menor o igual a n~~

H.I. ~~$0 \leq \text{len}(S) \leq n \Rightarrow \text{Mochila}(P, S)$~~ resuelve el problema. $= P(n)$

Tomo $Q \vee q: P(n) \Rightarrow P(n+1)$

Tome $\text{len}(S) = n+1$ entonces me fijo en el caso recursivo de la función. Hacer la mochila para un conjunto S es igual a hacer la mochila para todos los elementos menos un elemento v y luego fijarse si ese elemento conviene agregarlo al conjunto S' o no.

Por ejemplo, tomo el primer elemento de S y tengo que $|S - \text{SCos}| = n \Rightarrow$ Por H.I tengo que $\text{Mochila}(P, S - \text{SCos})$ devuelve el máximo valor que cumple con las condiciones del problema.

Ahora tengo que fijarme en el elemento SCos. Tengo dos opciones: 1) lo agrego y sumo su valor al valor máximo y resto su peso a P, $(\text{Mochila}(P, S - \text{SCos}) + \text{val})$ 2) no lo agrego y me quedo con el máximo valor y el peso que tengo hasta ahora.

(Acá se evidencia más que es indistinto del orden, ya que para cada elemento en algún punto tomo alguna de las dos decisiones).

Si $SE[0] = (\text{peso}, \text{valor})$:

El caso 1) sería ~~Mochila(P, S)~~ $Mochila(P - \text{peso}, S - SE[0]) + \text{valor}$
y el 2) es $Mochila(P, S - SE[0])$.

$S - SE[0]$ se puede escribir como $SE[1:]$.

Como queremos quedarnos con el máximo valor,
entonces quiero el $\max$ entre ambas opciones (la opción
que me devuelva un número mayor. Esto es:

$\max(Mochila(P, SE[1:]), Mochila(P - \text{peso}, SE[1:]) + \text{valor})$
y esto es exactamente lo que hace la función.

Entonces $P(n+1)$ también vale y por inducción, queda
probado para cualquier S (con cualquier longitud positiva).

Como para todos los casos la función resuelve el
problema $\Rightarrow$ También lo resuelve particularmente
para los P y S del enunciado original $\Rightarrow$

$Mochila(P, S)$ devuelve la respuesta al problema
plantado. ✓